

EXPERIMENT NO. _____

Exploring Pro-Code Extension of a Zoho Creator Application Using Zoho Catalyst / Custom Functions

Name: Karpaga Sundari V

Roll Number: 241401037

Course: _____

Date: _____

1. Aim

To explore how a Zoho Creator application can be extended with code. The work uses Creator custom functions (Deluge) and Zoho Catalyst functions, and shows when to use each.

2. Procedure

The steps use the example app *Bank Loan Application*. The app and names are examples. Menu names and settings may differ in your account.

Step 1. Take the example app: a loan application form built in Zoho Creator.

Step 2. Pick a need that is hard to meet with simple low-code tools. Here it is a detailed risk check on each loan.

Step 3. Decide where each piece of logic lives. Small, reusable logic stays in a Creator custom function (Deluge). Heavy or special logic goes to a Catalyst function.

Step 4. In Zoho Catalyst, create a project and a function. Pick a language the platform supports, such as Node.js, Java or Python.

Step 5. Write the risk rule in the function and deploy it from the Catalyst console or the Catalyst command-line tool.

Step 6. Note the function's web address and how calls to it are approved (its sign-in method).

Step 7. In Creator, set up a safe way to call it, such as a connection. Do not put keys or passwords inside the code.

Step 8. In the form workflow, call the Catalyst function with Deluge when the form is submitted.

Step 9. Show the result (for example, the risk level) on the record. Show a clear message if the call fails.

Step 10. Test with sample data in a non-live setting. Check the logs on both the Creator side and the Catalyst side.

Step 11. Review security and upkeep: least access, secrets kept out of code, monitoring, and who owns the function.

3. Output

No screenshots were supplied for this report. The items below are **illustrative diagrams, tables and code** that explain the idea. They are not results from a live test, and the code has not been run.

3.1 How the two parts work together

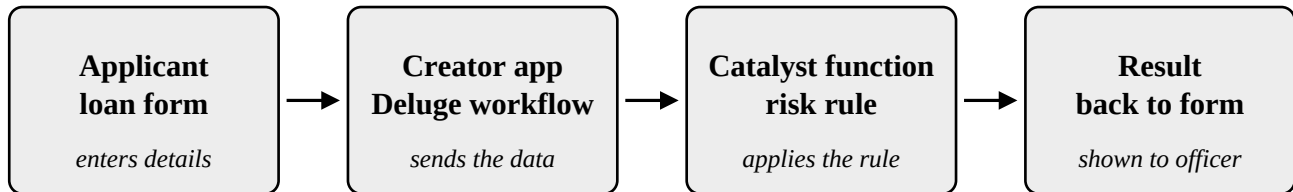


Figure 3.1: A Creator form calls a Catalyst function and receives a result (illustrative diagram).

3.2 Comparing the two ways

Point	Creator custom function	Catalyst function
Where it runs	Inside Zoho Creator.	On Zoho Catalyst, outside the Creator app.
Language	Deluge.	A general language such as Node.js, Java or Python (verify the current list).
Best for	Small, reusable logic used in many workflows.	Heavy, complex or special logic, or work with other systems.
Skills needed	Low-code skills.	Programming skills.
Extra setup	Little or none.	A project, deployment, and a way to approve calls.
Limits and cost	Check the Creator documentation.	Check the Catalyst documentation and pricing.

Table 3.2: Custom function and Catalyst function compared (general points; verify limits yourself).

3.3 Flow of one request

Step	Who	What happens
1	Applicant	Fills in and submits the loan form.
2	Creator workflow	Collects the needed fields and calls the Catalyst function.
3	Catalyst function	Applies the risk rule and returns a result.
4	Creator workflow	Saves the result on the record, or shows an error message.
5	Loan officer	Sees the result and decides on the loan.

Table 3.3: Steps of one loan request (illustrative flow).

3.4 Example call from Creator (Deluge)

```
// ILLUSTRATIVE EXAMPLE - not tested.
// Names, URL and connection are placeholders.
payload = Map();
payload.put("income", input.Monthly_Income);
payload.put("loan_amount", input.Loan_Amount);
response = invokeUrl
[
  url : "https://<your-catalyst-function-url>"
  type : POST
  parameters : payload.toString()
  connection : "<your_connection_name>"
];
input.Risk_Level = response.get("risk_level");
```

Code 3.4: Illustrative Deluge call to a Catalyst function.

3.5 Example Catalyst function (Node.js)

```
// ILLUSTRATIVE EXAMPLE - not tested.
// The rule below is a simple example, not a real bank rule.
const express = require("express");
const app = express();
app.use(express.json());

app.post("/", (req, res) => {
  const { income, loan_amount } = req.body;
  const ratio = loan_amount / (income * 12);
  const level = ratio > 0.5 ? "High" : "Low";
  res.json({ risk_level: level });
});

module.exports = app;
```

Code 3.5: Illustrative Catalyst function that returns a risk level.

3.6 Good practices for pro-code extensions

Concern	Good practice
Secrets and keys	Keep them in a connection or a secure setting, never in the code.
Errors	Handle a failed call and show a clear message to the user.
Testing	Test with sample data before real users, and read the logs on both sides.
Access	Give each function and user only the access that is needed.
Upkeep	Record who owns the function and keep its code in version control.
Cost and limits	Read the current documentation and pricing for both products.

Table 3.6: Good practices (general guidance).

4. Final Result

The pro-code extension of a Zoho Creator application was explored and documented. Small reusable logic can stay in Creator custom functions. Heavy or special logic can move to a Catalyst function that Creator calls. The diagrams and code are illustrative and were not run or tested.

2116241401037